

Sonatype Nexus for Artifact Storage

3 hands-on labs: standing up Nexus, configuring proxy/group repos, and publishing artifacts

LABS IN THIS GUIDE

- Lab 1 — Install Nexus and create hosted, proxy & group repositories
- Lab 2 — Point a Maven build at Nexus as a proxy for Maven Central
- Lab 3 — Publish a release artifact and a snapshot to Nexus hosted repos

Student lab guide · Companion to "Sonatype Nexus for Artifact Storage" (Part 2 of 3)

1 Lab 1: Install Nexus & Build the Repository Structure

Objective: Stand up a local Nexus instance and recreate the group/hosted/proxy structure described in the source document (`maven-public` group over `maven-releases` , `maven-snapshots` , and a Maven Central proxy).

Prerequisites

- Docker installed
- 4GB+ free RAM for the Nexus container

Steps

Step 1 — Run Nexus in Docker

```
docker run -d -p 8081:8081 --name nexus sonatype/nexus3
```

Wait 1–2 minutes for startup, then browse to `http://localhost:8081` .

Step 2 — Retrieve the generated admin password and log in

```
docker exec nexus cat /nexus-data/admin.password
```

Log in as `admin` with that password, then set a new password when prompted.

Step 3 — Confirm the default Maven proxy repository exists

Go to **Server administration > Repositories**. Nexus ships with a `maven-central` proxy repository by default — confirm it's listed and its remote storage URL points to `repo1.maven.org` .

Step 4 — Create the two hosted repositories

Create **Repository > Create repository > maven2 (hosted)** twice:

- Name: `maven-releases` , Version policy: **Release**, Deployment policy: **Disable redeploy**
- Name: `maven-snapshots` , Version policy: **Snapshot**, Deployment policy: **Allow redeploy**

Step 5 — Create the group repository

Create **Repository > Create repository > maven2 (group)**, name it `maven-public` , and add all three repositories (`maven-releases` , `maven-snapshots` , `maven-central`) as its members, in that order.

Verification Checklist

- [] `maven-public` group repository exists and lists all 3 members
- [] Browsing `maven-public` shows content proxied from Maven Central
- [] Student can explain the difference between "release" and "snapshot" deployment policy

Discussion question: Why put `maven-releases` and `maven-snapshots` ahead of `maven-central` in the group's member order? (Answer: Nexus checks group members in order, so internal artifacts should resolve before falling through to the public proxy.)

2 Lab 2: Use Nexus as a Maven Proxy

Objective: Reconfigure a real Maven build to resolve every dependency through the Nexus group repository built in Lab 1, instead of hitting Maven Central directly.

Prerequisites

- Completed Lab 1 (Nexus running with `maven-public` group)
- Any Maven project (e.g. reuse `maven-lifecycle-lab` from Part 1's lab guide)

Steps

Step 1 — Create a Nexus role and user for read access

In Nexus, create a user with the built-in `nx-repository-view-*-browse` and `read` privileges (or reuse the default `anonymous` access for this lab if enabled).

Step 2 — Configure Maven's global `settings.xml` to mirror everything through Nexus

```
<mirror>
  <id>nexus</id>
  <mirrorOf>*</mirrorOf>
  <url>http://localhost:8081/repository/maven-public/</url>
</mirror>
```

Place this inside `<mirrors>` in `~/.m2/settings.xml`.

Step 3 — Clear the local cache and rebuild

```
rm -rf ~/.m2/repository
mvn clean install
```

Step 4 — Confirm traffic went through Nexus, not the public internet

In the Nexus UI, go to **maven-central** > **Browse** and confirm the dependencies from your build now appear as cached components.

Verification Checklist

- [] Build completes successfully with the mirror configured
- [] Dependencies used by the build are visible as cached components inside the Nexus **maven-central** proxy
- [] Student can explain why this setup improves build resilience and speed for a whole team

Stretch goal: Disconnect from the internet (or block outbound traffic) and re-run `mvn clean install` using only the Nexus cache. Confirm the build still succeeds using previously cached artifacts.

3 Lab 3: Publish Artifacts to Nexus

Objective: Publish a release build and a snapshot build to the hosted repositories created in Lab 1, and observe the immutability difference between them.

Prerequisites

- Completed Labs 1 & 2

Steps

Step 1 — Add distributionManagement to pom.xml

```
<distributionManagement>
  <repository>
    <id>nexus-releases</id>
    <url>http://localhost:8081/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>nexus-snapshots</id>
    <url>http://localhost:8081/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

Step 2 — Add matching server credentials

```
<server>
  <id>nexus-releases</id>
  <username>admin</username>
  <password>YOUR_PASSWORD</password>
</server>
<server>
  <id>nexus-snapshots</id>
  <username>admin</username>
  <password>YOUR_PASSWORD</password>
</server>
```

Place both inside `<servers>` in `~/.m2/settings.xml`.

Step 3 — Publish a snapshot version

Set the version in `pom.xml` to `1.0.0-SNAPSHOT`, then:

```
mvn deploy
```

Confirm it appears under `maven-snapshots` in the Nexus UI. Deploy it a second time and confirm it **overwrites** the previous snapshot.

Step 4 — Publish a release version

Change the version to `1.0.0` (no `-SNAPSHOT` suffix), then:

```
mvn deploy
```

Confirm it appears under `maven-releases`. Attempt to run `mvn deploy` again with the same version number and confirm Nexus **rejects** the redeploy.

Verification Checklist

- ☐ Snapshot artifact successfully deployed and re-deployed (overwritten) in `maven-snapshots`
- ☐ Release artifact successfully deployed once to `maven-releases`
- ☐ Second deploy attempt of the same release version is rejected
- ☐ Student can explain why releases must be immutable in a production setting

Discussion question: What real-world problem does immutability of release artifacts prevent? (Answer: it guarantees that "version 1.0.0" means the exact same binary everywhere, forever — critical for reproducible deployments and audits.)